

• Quais são as entidades (classes) que devem ser criadas?

Para mapear este domínio de forma eficiente usando Orientação a Objetos, devemos criar as seguintes estruturas:

1. **Aluno (Entidade)**: Classe que representa o usuário consumidor da plataforma, contendo seus dados de identificação (ID, Nome, Email).
2. **Curso (Entidade)**: Classe que representa o produto educacional. Ela agrupa as aulas e carrega as definições de modelo de negócio (se é gratuito ou pago e seu respectivo valor).
3. **Aula (Entidade/Objeto de Valor)**: Classe que representa o conteúdo digital individual pertencente a um curso, contendo título e tempo de duração.
4. **Matricula (Entidade / Raiz de Agregação)**: A classe mais importante deste cenário. Ela serve como o "elo" de ligação entre o **Aluno** e o **Curso**. É ela quem deve gerenciar o histórico de progresso e a situação financeira do aluno naquele treinamento.
5. **TipoCurso (Enum)**: Enumerador para restringir as categorias de oferta: **Gratuito** e **Pago**.
6. **FormaPagamento (Enum)**: Enumerador para limitar as opções de transação exigidas: **Cartao** e **Pix**.

• Quais dessas entidades possuem ações (métodos) e quais são eles?

Para garantir o encapsulamento e evitar que a lógica de negócios fique espalhada fora das classes corretas, os comportamentos foram distribuídos assim:

1. **Na classe Curso**:
 - **AdicionarAula(Aula aula)**: Permite injetar novos conteúdos à grade curricular do curso de forma controlada.
2. **Na classe Matricula**:
 - **ProcessarPagamento(FormaPagamento forma)**: Recebe a modalidade financeira e altera o estado da matrícula para "Pago", liberando o acesso ao conteúdo.
 - **RegistrarProgresso(string aulaId)**: Adiciona o identificador de uma aula assistida à lista de conclusões do aluno, fazendo o controle do histórico.
 - **CalcularPercentualProgresso()**: Método matemático que divide a quantidade de aulas concluídas pelo total de aulas que o curso possui, retornando a evolução do aluno em formato de porcentagem (ex: 50.00%).

• Há necessidade de criação de interface? Se sim, descreva os comportamentos que devem ser criados.

Sim, pensando em um sistema comercial escalável, a introdução de interfaces resolve problemas críticos de integração e manutenção:

1. **IProcessadorPagamento**: Indispensável para o ecossistema de cursos pagos. A plataforma não deve se acoplar a um banco específico ou API direta. Essa interface permite que o sistema envie os dados de cobrança para qualquer operadora de cartão ou gerador de Pix (como Stone, Stripe ou Mercado Pago).

- *Comportamento:* `bool ExecutarCobranca(decimal valor, FormaPagamento forma);`
- 2. **INotificadorProgresso (Opcional, mas recomendável):** Uma interface para disparar eventos toda vez que o aluno avançar no curso, permitindo enviar e-mails de parabenização ou emitir o certificado automaticamente ao atingir 100%.
 - *Comportamento:* `void EnviarAlertaConclusao(Aluno aluno, Curso curso);`

• Quais são os relacionamentos entre as classes e entidades criadas?

O cenário descreve uma teia de relações com diferentes multiplicidades:

1. **Muitos para Muitos (\$N:N\$) entre Aluno e Curso resolvido por Matricula:** Um aluno pode fazer vários cursos e um curso tem vários alunos. Na orientação a objetos, quebramos essa relação complexa fazendo com que a `Matricula` conheça um `Aluno` e um `Curso`. Assim, a plataforma gerencia uma lista de Matrículas.
2. **Composição (1 para Muitos / \$1:N\$) entre Curso e Aula:** Um curso possui várias aulas. É uma relação de dependência forte: a aula pertence exclusivamente àquele curso e, se o curso for deletado do sistema, as aulas dele deixam de existir.
3. **Associação de Atributo:** O `Curso` usa o enum `TipoCurso` para definir suas regras de preço, e a `Matricula` usa o enum `FormaPagamento` para registrar o fechamento financeiro.

• Onde o desenvolvedor deve tratar exceções em caso de críticas no processo?

As validações e tratamentos devem seguir a separação de responsabilidades em camadas:

1. **Na Camada de Domínio (Regras de Negócio na classe Matricula):** O desenvolvedor deve lançar exceções (`InvalidOperationException` ou `ArgumentException`) imediatamente se as regras de uso da plataforma forem violadas:
 - Bloquear e lançar erro se o método `ProcessarPagamento` for chamado para um curso que foi cadastrado como `Gratuito`.
 - Bloquear e lançar erro no método `RegistrarProgresso` se o aluno tentar assistir a uma aula de um curso Pago que ele ainda não pagou.
 - Bloquear e lançar erro se o ID da aula que está sendo registrada como "concluída" sequer pertencer à grade curricular daquele curso.
2. **Na Camada de Aplicação / Serviços de Infraestrutura (Gateways de Pagamento):** O desenvolvedor deve tratar exceções de infraestrutura (como quedas de internet ou timeouts) usando blocos `try-catch` na integração do Cartão/Pix. Se a API do Pix estiver fora do ar, o sistema captura essa falha de conexão, gera um log interno com o erro técnico real e avisa amigavelmente na tela do aluno: *"Sistema de pagamento temporariamente indisponível. Por favor, tente novamente em instantes."*